THE UNIVERSITY OF AZAD JAMMU AND KASHMIR, MUZAFFARABAD



COURSE TITLE	Digital Imaging Processing
COURSE CODE	CS-3112
LAB TITLE	Python Setup, Fundamentals & Imports
STUDENT NAME	MAHDI ALI NAHVI
STUDENT ROLL NUMBER	2024-SE-16
INSTRUCTOR NAME	Engr. Fahad Nisar
SUBMISSION DATE	October 23, 2026
LAB NUMBER	01

1. Objective

The objective of this lab was to set up a working Python development environment on a Windows computer and to practise the basic building blocks of the language that will be used throughout the Digital Image Processing course. This included installing Python and Visual Studio Code, running a first program, solving six exercises on variables, conditions, loops, functions and collections, installing the NumPy, Pillow and Matplotlib packages, and verifying the import statements that later labs depend on.

2. Installation and Setup

Step 1: Installing Python

I opened the official Python downloads page at python.org/downloads in my browser. The page detected Windows and offered the Python install manager as the main download, with a link to a standalone installer for Python 3.15.0 below it. I used this page to download and install Python 3 on my computer, then closed and reopened the terminal so that the new installation was picked up.

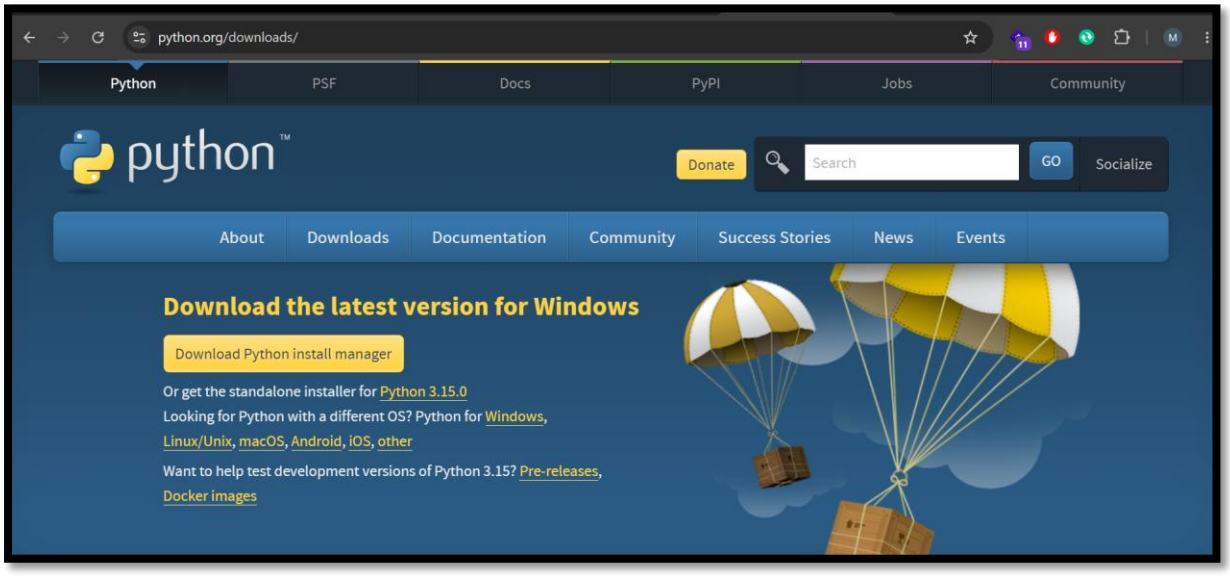


Figure 1: Python downloads page (python.org/downloads) offering the Windows installer

Step 2: Verifying Python and pip

I opened a new Command Prompt window and ran two commands. The first, python --version, confirmed that Python is available from the command line and reported Python 3.12.10. The second, python -m pip --version, confirmed that pip is installed and linked to this interpreter. It reported pip 25.0.1 located in the Python312 folder under my user profile (AppData\Local\Programs\Python), tied to Python 3.12.

```
C:\Users\HP>python --version
Python 3.12.10

C:\Users\HP>python -m pip --version
pip 25.0.1 from C:\Users\HP\AppData\Local\Programs\Python\Python312\Lib\site-packages\pip (python 3.12)
```

Figure 2: Python and pip version checks in Command Prompt

Step 3: Installing VS Code and the Python extension

Next I visited code.visualstudio.com and used the “Download for Windows” button to get Visual Studio Code, which I then installed. After opening VS Code, I went to the Extensions view (Ctrl + Shift + X), searched for “Python” and installed the Python extension published by Microsoft. The screenshot shows the extension page with the status “Installing”, along with its description: language support for IntelliSense, debugging, linting, formatting, refactoring and unit tests.

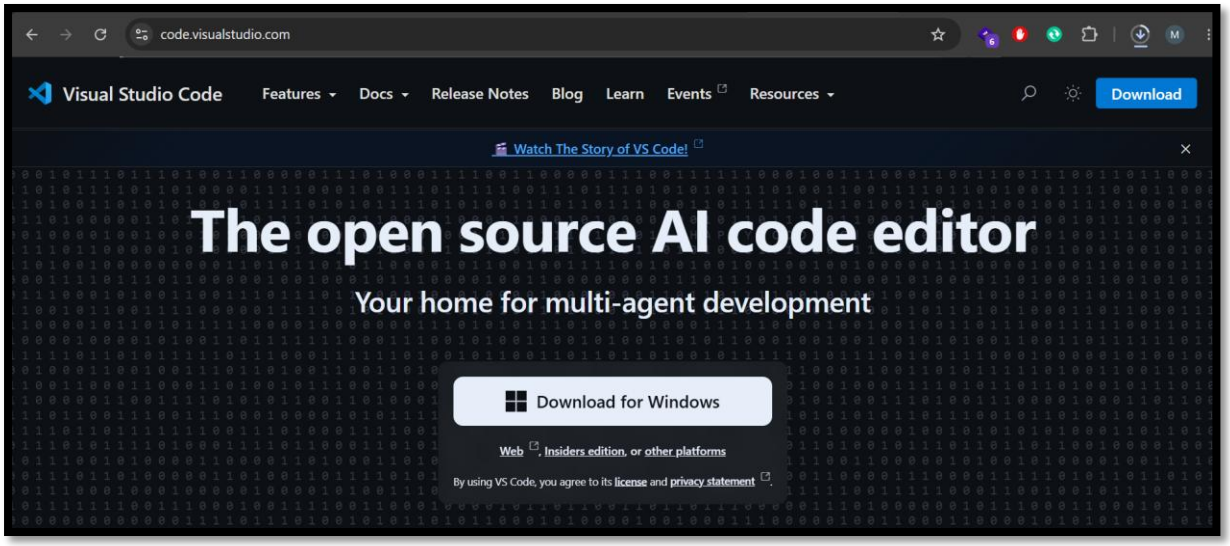


Figure 3: Visual Studio Code download page



Figure 4: Installing the Python extension by Microsoft from the VS Code Extensions Marketplace

To select the interpreter, I pressed Ctrl + Shift + P in VS Code and chose “Python: Select Interpreter”. From the list I picked Python 3.12 (64-bit), located at AppData\Local\Programs\Python\Python312\python.exe and marked “Global”. This is the same installation that was verified in Step 2, so VS Code now runs my code with the correct Python version.

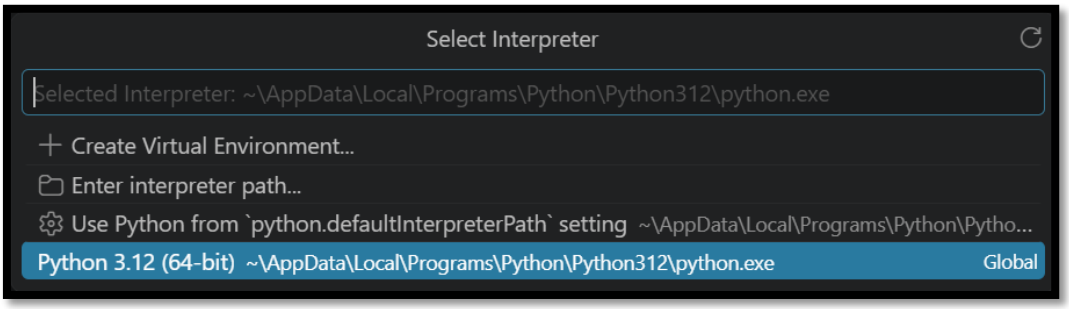


Figure 5: Selecting the Python 3.12 interpreter in VS Code

Step 4: Running the first Python program

I created a project folder for the lab (Lab 01, inside my Digital Image Processing labs folder) and created a file named hello.py in VS Code. The file contains a comment on line 1 (comments start with #) and a print statement on line 2 that displays the message “Hello, Digital Image Processing!”.

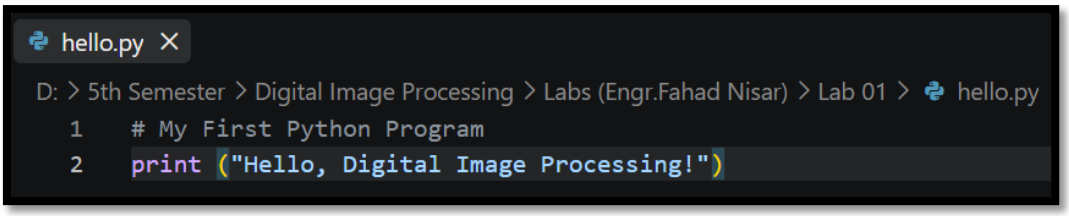


Figure 6: Source code of hello.py in VS Code

I then opened Command Prompt inside the Lab 01 folder and ran python hello.py. The terminal printed Hello, Digital Image Processing!, which matches the expected output and confirms that Python, the project folder and the file are all working correctly.

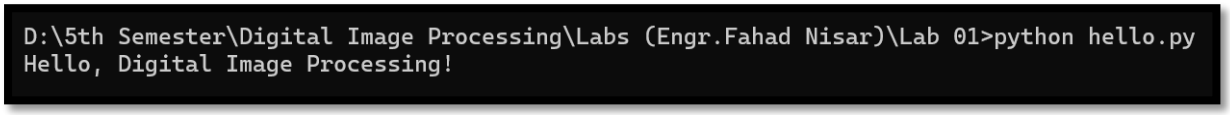


Figure 7: Running hello.py from Command Prompt and its output

3. Basic Python Exercises (basics.py)

All six exercises were written in a single file named basics.py in the same folder. Each section is labelled with a comment, uses four-space indentation, and prints its results. I ran the file with python basics.py after completing each exercise.

Exercise 1: Variables and Data Types

I created four variables: width = 640 and height = 480 (integers), image_name = "sample.png" (a string) and is_color = True (a boolean). Each variable was printed together with its type using the type() function, and the results were class int, int, str and bool respectively. I then calculated the total number of pixels with total_pixels = width * height and printed it. The output was 307200, which matches the expected value (640 × 480).

basics.py

D: > 5th Semester > Digital Image Processing > Labs (Engr.Fahad Nisar) > Lab 01 > basics.py > ...

```python
# Exercise 01 : Variables and Data Types

# Define variables
width = 640
height = 480
image_name = "sample.png"
is_color = True

# Print the variables and their types
print("Width:", width, "Type:", type(width))
print("Height:", height, "Type:", type(height))
print("Image Name:", image_name, "Type:", type(image_name))
print("Is Color:", is_color, "Type:", type(is_color))

# Calculate the total number of pixels
total_pixels = width * height

# Print the final calculated result
print("Total number of pixels: ", total_pixels)
```

Problems Output Debug Console Terminal Ports

```
PS D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 01> python basics.py
Width: 640 Type: <class 'int'>
Height: 480 Type: <class 'int'>
Image Name: sample.png Type: <class 'str'>
Is Color: True Type: <class 'bool'>
Total number of pixels:  307200
```

Figure 8: Exercise 1 code and terminal output

Exercise 2: if, elif and else

I stored the test values -1, 0, 127, 128, 255 and 256 in a list and looped over them. For each intensity, the if branch checks whether the value is below 0 or above 255 and labels it “Invalid”. The elif branch labels a valid value of 128 or above as “Bright pixel”, and the else branch labels everything else as “Dark pixel”. The order of the checks matters: the invalid range is tested first so that out-of-range values never reach the brightness test. The output was Invalid, Dark pixel, Dark pixel, Bright pixel, Bright pixel, Invalid, which matches the expected labels, including both boundary values (127/128 and 255/256).

basics.py

D: > 5th Semester > Digital Image Processing > Labs (Engr.Fahad Nisar) > Lab 01 > basics.py > ...

```python
# Exercise 2: if, elif, and else - Pixel Intensity Labeler

# Define the list of test intensities
test_intensities = [-1, 0, 127, 128, 255, 256]

# Loop through each intensity to test the conditions
for intensity in test_intensities:
    # Check if the intensity is out of bounds
    if intensity < 0 or intensity > 255:
        label = "Invalid"
    # Check if the valid intensity is 128 or above
    elif intensity >= 128:
        label = "Bright pixel"
    # Otherwise, it must be lower than 128
    else:
        label = "Dark pixel"

    # Print the intensity and its resulting label
    print(f"Intensity {intensity}: {label}")
```

Problems Output Debug Console Terminal Ports

```
PS D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 01> python basics.py
Intensity -1: Invalid
Intensity 0: Dark pixel
Intensity 127: Dark pixel
Intensity 128: Bright pixel
Intensity 255: Bright pixel
Intensity 256: Invalid
```

Figure 9: Exercise 2 code and labels for all six test values

Exercise 3: for and while Loops

For the for loop, I defined pixels = [20, 130, 255] and a threshold of 128. The loop visits each value and prints “Bright pixel” if the value is at least the threshold, otherwise “Dark pixel”. The output was 20: Dark pixel, 130: Bright pixel, 255: Bright pixel.

For the while loop, I started a counter at 0 and used the condition counter < 3. Inside the loop the counter is printed and then increased with counter += 1. Updating the counter is essential, because without it the condition would never become false and the loop would run forever. The loop printed 0, 1 and 2 and stopped before 3, as required.

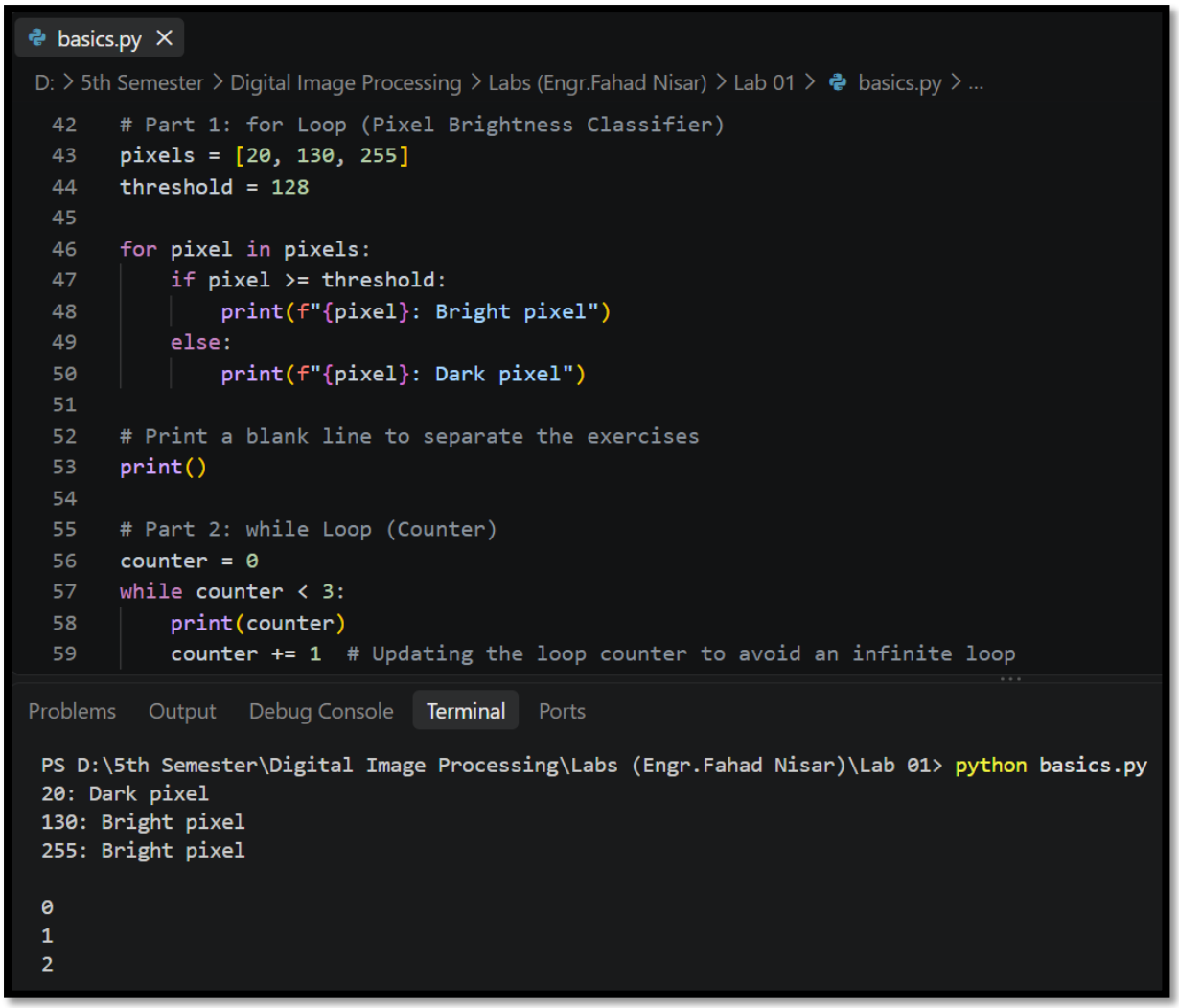


Figure 10: Exercise 3 for-loop and while-loop code with output

Exercise 4: Functions

I defined a function is_bright(pixel, threshold=128) that returns the result of the comparison pixel >= threshold, so it returns True when the pixel is at least the threshold and False otherwise. The threshold parameter has a default value of 128, which is used when the caller does not supply one. I tested it with three calls: is_bright(127) gave False, is_bright(128) gave True, and is_bright(200, 210) gave False because the custom threshold of 210 overrides the default. These results match the expected output.

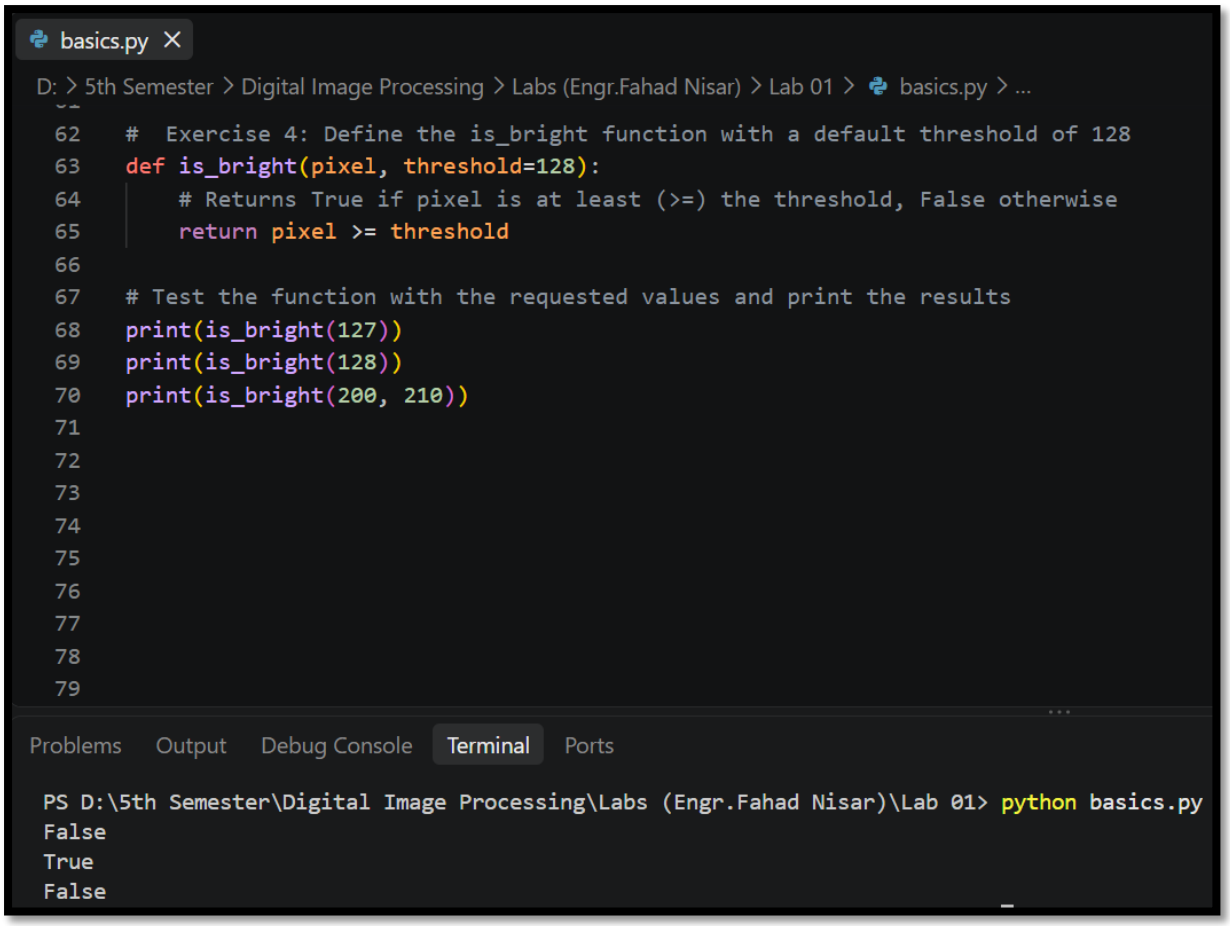


Figure 11: Exercise 4 function definition, test calls and output

Exercise 5: Lists and Tuples

For the list part, I created pixels = [0, 128, 255] and printed its first item (index 0), which gave 0. I replaced the middle item with 150 using pixels[1] = 150 and added 64 to the end with pixels.append(64). Printing the list gave [0, 150, 255, 64] and its length was 4, as expected.

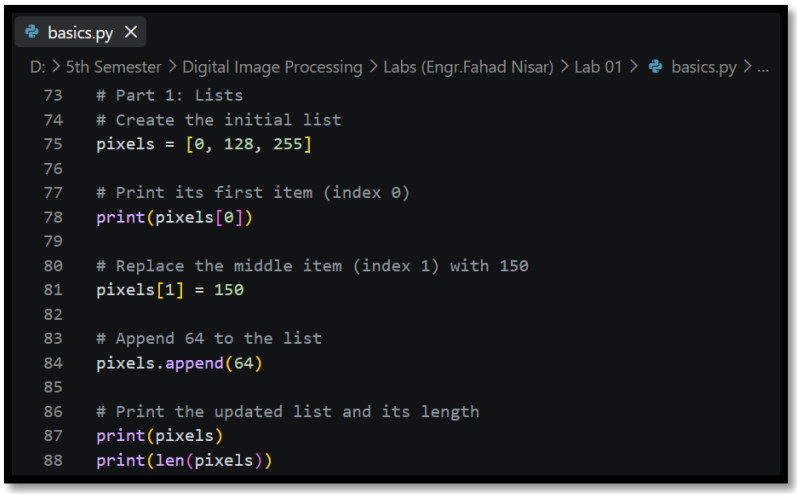


Figure 12: Exercise 5 (Part 1) list operations

For the tuple part, I created size = (640, 480) and unpacked it into two variables with width, height = size. Printing them gave 640 and 480. In a comment I explained that lists are mutable, so their contents and size can be changed in place, while tuples are immutable, so once created their elements are fixed. This is why pixels[1] = 150 works on a list but the same assignment on a tuple would raise an error.

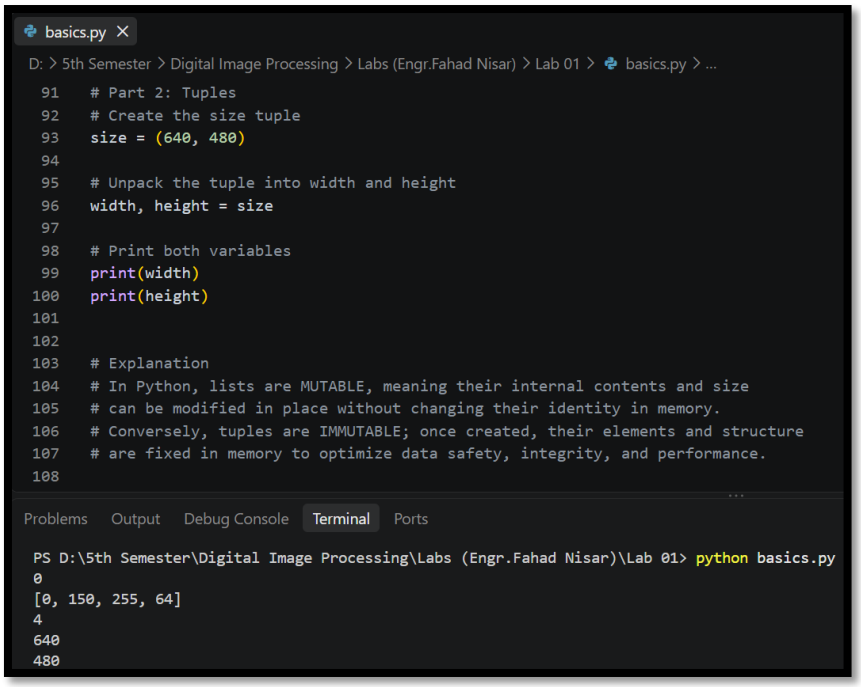


Figure 13: Exercise 5 (Part 2) tuple unpacking, mutability explanation and combined terminal output

Exercise 6: Dictionaries and Sets

For the dictionary part, I created image = {"width": 640, "height": 480} and added a new key-value pair with image["mode"] = "RGB". I then printed the values by key, which gave 640 for the width and RGB for the mode. For the set part, I started with the list ["cat", "dog", "cat", "bird"] and converted it with set(). A set keeps only unique items, so the duplicate “cat” was removed and len() returned 3, which is the expected number of unique labels.

basics.py

D: > 5th Semester > Digital Image Processing > Labs (Engr.Fahad Nisar) > Lab 01 > basics.py > ...

```python
# Part 1: Dictionaries
# Create the initial image dictionary
image = {"width": 640, "height": 480}

# Add the new key-value pair "mode": "RGB"
image["mode"] = "RGB"

# Print the width and mode using their respective keys
print(image["width"])
print(image["mode"])

# Part 2: Sets
# Create the initial list containing a duplicate item ("cat")
labels_list = ["cat", "dog", "cat", "bird"]

# Convert the list into a set to automatically strip out duplicates
labels_set = set(labels_list)

# Print the length of the set
print(len(labels_set))
```

Problems Output Debug Console Terminal Ports

```
PS D:\5th Semester\Digital Image Processing\Labs (Engr.Fahad Nisar)\Lab 01> python basics.py
640
RGB
3
```

Figure 14: Exercise 6 dictionary and set code with output

4. Installing Packages and Verifying Imports

Step 6: Installing the required packages

The required packages were installed from Command Prompt using pip. NumPy supports numerical arrays, Pillow provides image tools and Matplotlib supports plotting. I installed them one at a time with python -m pip install numpy, python -m pip install pillow and python -m pip install matplotlib, and waited for each installation to finish.

NumPy: pip collected and downloaded the NumPy 2.5.4 wheel (12.6 MB) and finished with "Successfully installed numpy-2.5.4".

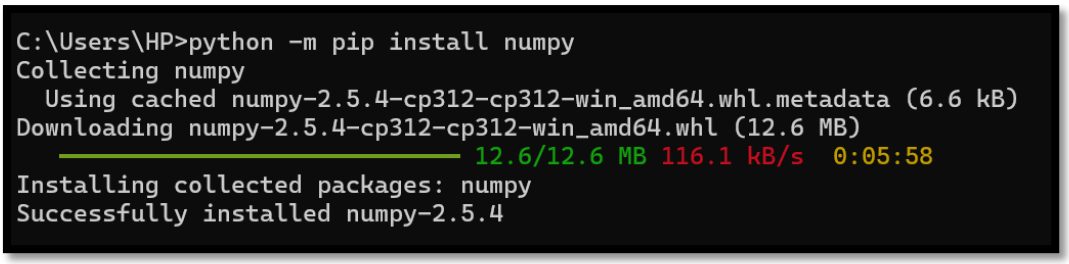


Figure 15: Installing NumPy with pip

Pillow: pip downloaded the Pillow 13.0.0 wheel (7.3 MB) and finished with "Successfully installed pillow-13.0.0".



Figure 16: Installing Pillow with pip

Matplotlib: pip downloaded Matplotlib 3.11.2 together with its supporting packages (contourpy, cycler, fonttools, kiwisolver and pyparsing). It reported that NumPy and Pillow were already satisfied because they had been installed in the previous steps, and ended with a "Successfully installed" message listing all the new packages including matplotlib-3.11.2.

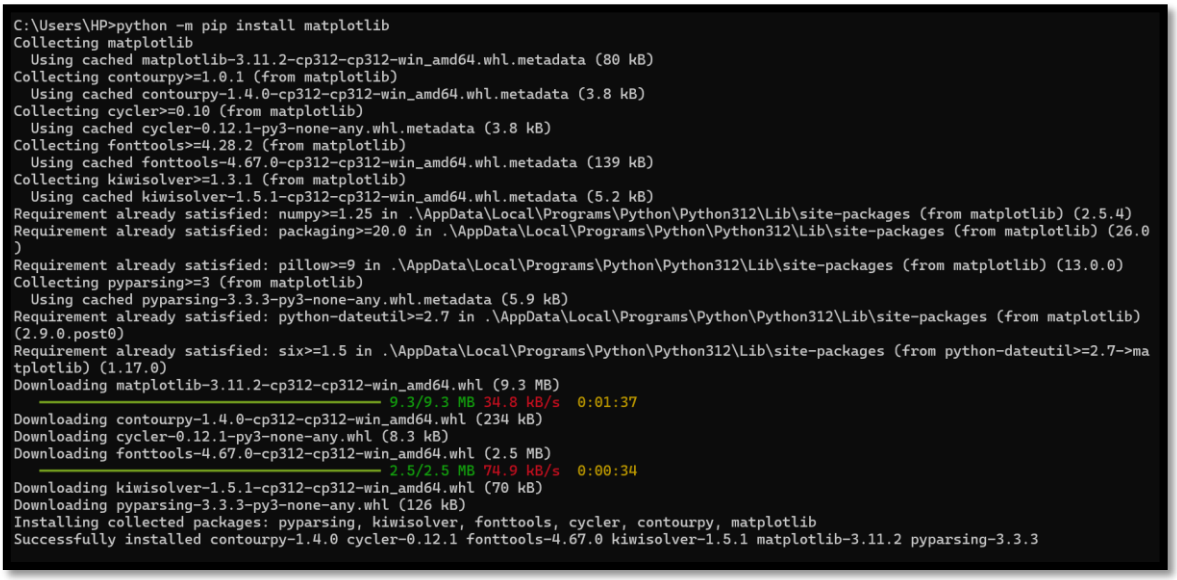


Figure 17: Installing Matplotlib with pip

To confirm the installation, I ran python -m pip show numpy pillow matplotlib. The output lists the name, version, summary, home page, author, license and install location for all three packages: numpy 2.5.4 (fundamental package for array computing), pillow 13.0.0 (Python Imaging Library fork) and matplotlib 3.11.2 (Python plotting package). All are installed in the Python312 site-packages folder. It also shows that NumPy is required by matplotlib, and Pillow is required by matplotlib.

```
C:\Users\HP>python -m pip show numpy pillow matplotlib
Name: numpy
Version: 2.5.4
Summary: Fundamental package for array computing in Python
Home-page: https://numpy.org
Author: Travis E. Oliphant et al.
Author-email:
License-Expression: BSD-3-Clause AND 0BSD AND MIT AND Zlib AND CC0-1.0
Location: C:\Users\HP\AppData\Local\Programs\Python\Python312\Lib\site-packages
Requires:
Required-by: contourpy, matplotlib
---
Name: pillow
Version: 13.0.0
Summary: Python Imaging Library (fork)
Home-page: https://python-pillow.org
Author:
Author-email: Jeffrey 'Alex' Clark <aclark@aclark.net>
License-Expression: MIT-CMU
Location: C:\Users\HP\AppData\Local\Programs\Python\Python312\Lib\site-packages
Requires:
Required-by: matplotlib
---
Name: matplotlib
Version: 3.11.2
Summary: Python plotting package
Home-page: https://matplotlib.org
Author: John D. Hunter, Michael Droettboom
Author-email: Unknown <matplotlib-users@python.org>
License: License agreement for matplotlib versions 1.3.0 and later
```

Figure 18: Output of pip show for numpy, pillow and matplotlib

Step 8: Writing and verifying the import statements

I created the import file in the Lab 01 project folder (named Lab01_Imports.py) and saved only three import statements: import numpy as np, from PIL import Image, and import matplotlib.pyplot as plt. I then ran it from the VS Code terminal in the same folder with python Lab01_Imports.py. The command finished and returned straight to the prompt with no error message and no printed output, which is the expected result because the file contains only imports. This confirms that NumPy, Pillow and Matplotlib are all installed correctly and can be imported.

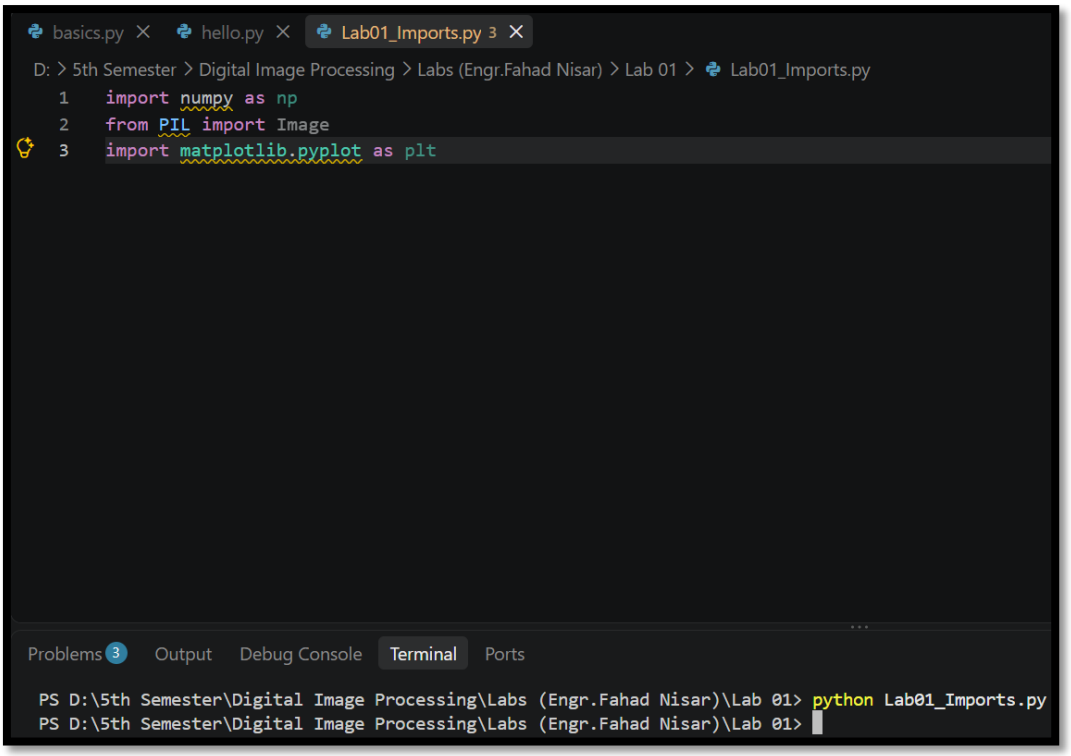


Figure 19: Saved import statements and successful run of the import file

Note: VS Code shows a Problems count and underlines on the import lines. Since the imports are not used anywhere in the file, these are editor hints rather than errors, and the program itself ran without any error.

5. Python versus pip, and Purpose of Imports

Python is the programming language and its interpreter, which reads and runs .py files such as hello.py and basics.py. pip is Python's package manager: it downloads and installs extra libraries like NumPy, Pillow and Matplotlib that are not part of the standard installation. Running it as python -m pip makes sure the packages go to the same Python installation that will run the code.

Import statements load an installed library into a program so its functions can be used. In this lab, NumPy (as np) supplies arrays for pixel data, Pillow's Image class opens and processes image files, and Matplotlib's pyplot (as plt) displays images and plots. A clean run of lab01_imports.py shows that all three libraries are ready for the upcoming image processing labs.

6. Conclusion

In this lab I installed Python and VS Code, verified Python and pip from Command Prompt, and ran my first program. I then solved all six exercises covering variables and data types, conditions, loops, functions, lists, tuples, dictionaries and sets, and every output matched the expected result. I also learned how packages are installed with pip and verified with import statements, which prepares my environment for the Digital Image Processing labs.

7. Files Submitted

- hello.py
- basics.py
- Lab01_Imports.py